

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-210БВ-25

Студент: Хабиров Р.И.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 23.09.26

Москва, 2026

Постановка задачи

Вариант 7.

Родительский процесс создает дочерний процесс. Первой строчкой пользователь в консоль родительского процесса вводит имя файла, которое будет использовано для открытия файла с таким именем на чтение. Стандартный поток ввода дочернего процесса переопределяется открытым файлом. Дочерний процесс читает команды из стандартного потока ввода. Стандартный поток вывода дочернего процесса перенаправляется в `pipe1`. Родительский процесс читает из `pipe1` и прочитанное выводит в свой стандартный поток вывода. Родительский и дочерний процесс должны быть представлены разными программами.

В файле записаны команды вида: «число число число<newline>». Дочерний процесс считает их сумму и выводит результат в стандартный поток вывода. Числа имеют тип `float`. Количество чисел может быть произвольным.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; – создает дочерний процесс.
- `int execve(const char *filename, char *const argv[], char *const envp[])` - замена образа памяти процесса
- `int pipe(pipe_fds[2])`; – создание неименованного канала для передачи данных между процессами
- `pid_t waitpid(pid_t pid, int *status, int options)` - Ожидание завершения дочернего процесса
- `int dup2(int oldfd, int newfd)` - переназначение файлового дескриптора
- `int open(const char *pathname, int flags, mode_t mode)` - открытие\создание файла
- `int close(int fd)` - закрыть файл

В рамках данной лабораторной работы я писал программу, которая в ходе своей работы создаёт дочерний процесс, которому необходимо считать числа типа `float` из файла, а затем передать их для чтения основному процессу, который эти числа выводит в поток вывода. Родительский и дочерний процессы представлены в виде двух отдельно скомпилированных программ. Запуск дочернего процесса происходит с помощью системного вызова `execve`. Передача данных между процессами организована при помощи системного вызова `pipe`, который создает неименованный канал между ними. При помощи системного вызова `dup2` происходит переназначение файлового дескриптора для дочернего процесса, и он считывает данные не из стандартного потока ввода, а из уже открытого файла, и передает данные не в стандартный поток вывода, а записывает их в конец канала для записи.

Код программы

Программа `child.c`

```
#include <math.h>
```

```
#include <unistd.h>
```

```

float parse_float(const char* str){
    float res = 0.0;
    float frac = 1.0;
    int i = 0;
    int is_frac = 0;
    int sign = 1;
    if (str[0] == '-') {
        sign = -1;
        i++;
    }
    for (; str[i] != '\0'; i++){
        if (str[i] == '.'){
            is_frac = 1;
            continue;
        }
        if (str[i] >= '0' && str[i] <= '9'){
            if (!is_frac){
                res = res * 10.0 + (str[i] - '0');
                continue;
            }
            else {
                frac = frac / 10;
                res = res + (str[i] - '0') * frac;
            }
        }
    }
    return res * sign;
}

```

```

int float_str(char buff[64], float num) {

```

```
int i = 0;
```

```
if (num < 0) {  
    buff[i++] = '-';  
    num = fabs(num);  
}
```

```
int res = (int)num;  
int int_frac = (int)((num - res) * 100.0f + 0.5f);
```

```
if (int_frac >= 100) {  
    res += 1;  
    int_frac -= 100;  
}
```

```
if (res == 0) {  
    buff[i++] = '0';  
}
```

```
else {  
    int temp_res = res;  
    int len_res = 0;  
    while (temp_res > 0) {  
        len_res++;  
        temp_res /= 10;  
    }
```

```
int temp_i = i + len_res - 1;  
while (res > 0) {  
    buff[temp_i--] = (res % 10) + '0';  
    res /= 10;
```

```

    }

    i += len_res;
}

buff[i++] = '.';

buff[i++] = (int_frac / 10) + '0';
buff[i++] = (int_frac % 10) + '0';

buff[i++] = '\n';
buff[i] = '\0';

return i;
}

int main(){
    char num_buff[64], ch;
    int num_len = 0, has_num = 0;
    float line_sum = 0.0f;
    while (read(0, &ch, 1) > 0){
        if (ch >= '0' && ch <= '9' || ch == '.' || (ch == '-' && num_len == 0)){
            if (num_len < 63){
                num_buff[num_len++] = ch;
            }
        }
        else{
            if (num_len > 0){
                num_buff[num_len] = '\0';
                line_sum += parse_float(num_buff);
                num_len = 0;
            }
        }
    }
}

```

```

        has_num = 1;
    }
    if (ch == '\n'){
        if (has_num){
            char out_buff[64];

            int out_len = float_str(out_buff, line_sum);

            write(1, out_buff, out_len);
        }

        line_sum = 0.0f;

        has_num = 0;
    }
}

if (num_len > 0){
    num_buff[num_len] = '\0';

    line_sum += parse_float(num_buff);

    num_len = 0;

    has_num = 1;
}

if (has_num){
    char out_buff[64];

    int out_len = float_str(out_buff, line_sum);

    write(1, out_buff, out_len);
}

return 0;
}

```

Программа parent.c

```
#include <unistd.h>
```

```
#include <fcntl.h>
```

```
#include <sys/wait.h>
```

```
#include <sys/types.h>
```

```
int str_len(const char *str){
```

```
    int len = 0;
```

```
    while (str[len] != '\0'){
```

```
        len ++;
```

```
    }
```

```
    return len;
```

```
}
```

```
int main(){
```

```
    char filename[256];
```

```
    const char *msg = "Введите имя файла: ";
```

```
    write(1, msg, str_len(msg));
```

```
    ssize_t bytes_read = read(0, filename, sizeof(filename) - 1);
```

```
    if (bytes_read <= 0){
```

```
        return 1;
```

```
    }
```

```
    filename[bytes_read] = '\0';
```

```
    for (int i = 0; i < bytes_read; i++){
```

```
        if (filename[i] == '\n'){
```

```
            filename[i] = '\0';
```

```
            break;
```

```
        }
```

```
    }
```

```
int pipe_fds[2];

if (pipe(pipe_fds) == -1){

    const char *msg = "Не удалось создать pipe\n";

    write(2, msg, str_len(msg));

    return 1;

}

pid_t pid = fork();

if (pid == -1){

    const char *msg = "Ошибка fork\n";

    write(2, msg, str_len(msg));

    return 1;

}

else if(pid == 0){

    int file_fd = open(filename, O_RDONLY);

    char *child_argv[] = {"/child", NULL};

    char *child_envp[] = {NULL};

    if (file_fd == -1){

        const char *msg = "Не удалось открыть файл\n";

        write(2, msg, str_len(msg));

        return 1;

    }

    dup2(file_fd, 0);

    close(file_fd);

    close(pipe_fds[0]);
```

```

dup2(pipe_fds[1], 1);
close(pipe_fds[1]);

execve("./child", child_argv, child_envp);

const char *msg = "Не удалось запустить ./child\n";
write(2, msg, str_len(msg));
return 1;
}

else{
    close(pipe_fds[1]);

    char buff[128];
    ssize_t n;

    const char *msg = "\nРезультаты из дочернего процесса\n";
    write(1, msg, str_len(msg));

    while ((n = read(pipe_fds[0], buff, sizeof(buff))) > 0){
        write(1, buff, n);
    }

    close(pipe_fds[0]);

    int status;
    waitpid(pid, &status, 0);
}

```

```
return 0;  
}
```

Протокол работы программы

```
flosy@Flosy:~$ cd 2_course/OS-labs/lab_1  
flosy@Flosy:~/2_course/OS-labs/lab_1$ gcc child.c -o child -lm  
flosy@Flosy:~/2_course/OS-labs/lab_1$ gcc parent.c -o parent  
flosy@Flosy:~/2_course/OS-labs/lab_1$ ./parent
```

Введите имя файла: f1

Результаты из дочернего процесса

7.00

30.50

```
flosy@Flosy:~/2_course/OS-labs/lab_1$ cat f1
```

1.5 2.5 3.0

10.2 20.3

```
flosy@Flosy:~/2_course/OS-labs/lab_1$ ./parent
```

Введите имя файла: f2

Результаты из дочернего процесса

10.90

0.95

```
flosy@Flosy:~/2_course/OS-labs/lab_1$ cat f2
```

-10.15 0.55 20.5

0.95

```
flosy@Flosy:~/2_course/OS-labs/lab_1$ ./parent
```

Введите имя файла: f3

Результаты из дочернего процесса

-0.40

5469.25

flosy@Flosy:~/2_course/OS-labs/lab_1\$ cat f3

9.99 0.123 -10.513

124.124 0.0 5345.124

Вывод

В ходе данной лабораторной работы я приобрел практические навыки в управлении процессами в ОС и в обеспечении обмена данными между процессами посредством каналов. Мною была составлена и отлажена программа на языке Си, осуществляющая работу с процессами и взаимодействие между ними в операционной системе Linux.